

PhD Assignment Report

Hashim Karim

h.karim-2@student.tudelft.nl
github.com/hashimkarim/densor

Approach

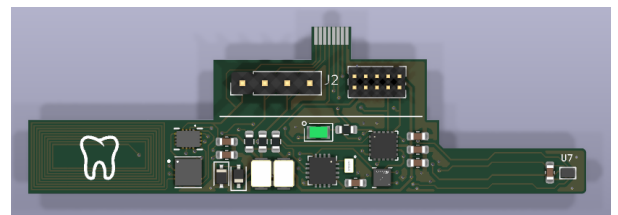
I extended Densor with external temperature sensing and independent sampling rates. Revision 1 (R1) corrects baseline logging; Revision 2 (R2) and Revision 3 (R3) introduce shared and partitioned multirate storage, respectively.

Sensor selection

I considered an NTC thermistor, MAX30208, TMP117 and TMP119 for the 2.6–1.8 V supply, prioritizing energy per reading. An NTC would require an accuracy budget covering the thermistor, reference resistor and ADC, and possibly calibration. [MAX30208](#) offered lower conversion current, but its longer possible result latency could extend the device's active time. I chose **TMP119** for factory-calibrated I²C readings, conversion energy similar to TMP117, tighter specified accuracy and better tolerance to assembly and bending stress. Professional assembly made its small package practical.

Sensor placement

I placed TMP119 14.5 mm beyond the original board edge, with its 100 nF decoupling capacitor nearby. The extension is intended to allow placement farther back in the mouth, away from the main electronics. Literature examples favoured posterior sites for reduced airflow effects ([de Almeida e Bueno et al., 2022](#)) and greater temperature stability ([Zhao and Bergmann, 2025](#)).



Top-view PCB render. TMP119 (U7) sits at the right-hand tip of the extension.

Firmware: temperature sensing and EEPROM logging

At each RTC wake, firmware reads the sensors due, saves their data and powers down. I added TMP119 **one-shot measurements without averaging** to shorten active time: a conversion typically takes **15.5 ms**, versus **124 ms** for eight averages. While TMP119 converts, firmware reads other due sensors, then collects its result with a timeout. Users can select either temperature sensor or both; the log keeps their readings separate, records supply voltage and reports missing sensors.

Extra EEPROM page writes cost energy. In R1, I moved the two-byte log pointer from addresses 7–8 to 8–9 and aligned records to four-byte page boundaries. This reduces avoidable page crossings at the cost of padding bytes. Firmware checks that the record fits, waits for its writes to finish, then saves the new pointer before powering off. If a write cannot be confirmed, it stops.

Firmware: multirate sampling

Independent sampling rates allow frequent motion measurements without spending energy on unnecessary temperature readings. The RTC wakes at the **GCD** of the sensor periods; the finite-state machine (FSM) repeats after their **LCM**. For temperature/photodiode/acceleration every 120/30/10 s, the RTC wakes every 10 s: acceleration runs each tick, photodiode every 3 ticks and temperature every 12 ticks. The cycle repeats after 12 ticks (120 s). The FSM keeps its step in RTC RAM between MCU power cycles and advances after saving the data.

R2 stores samples together in shared memory; **R3** gives each sensor its own region, sized in Android from record size and sampling rate and fixed during a recording. Suffix **a** uses the retained FSM; **b** calculates the sampling step from RTC time. Keeping the current write positions in RTC RAM reduces EEPROM updates. Periodic EEPROM checkpoints save these positions for recovery. Checksums help detect damaged records; loss of timing stops the recording.

Android integration and visualization

One Android app supports **R1**, **R2a**, **R2b**, **R3a** and **R3b**, with common-rate or per-sensor settings. Settings take effect after firmware confirmation; checks detect changes during NFC reads. Binary/CSV exports and temperature plots use each sensor's sampling times.

I built **Sampling lab** to compare selected rates against synthetic 100 Hz signals and export capacity-checked EEPROM images for Android debug import. **EEPROM lab** compares records, padding, page crossings and allocation across baseline/R1/R2/R3, with adjustable payloads and extra sensors for packing experiments. These are simulations, not energy measurements.

Recorded checks include all four builds, firmware fault injection, 360 C-generated images decoded in Java and 14 Android fixture tests on an S21 FE. Energy savings, unaveraged temperature accuracy and physical power-loss behavior still require board measurements.

AI usage. I used AI for minor KiCad tasks (adding 3D models and exporting files), literature review, repository documentation, adapting the Android app to the firmware using my open-source Android AI development tool, [Android Agent Lab](#), and creating web apps to visualize EEPROM paging and sampling.